

Ecto

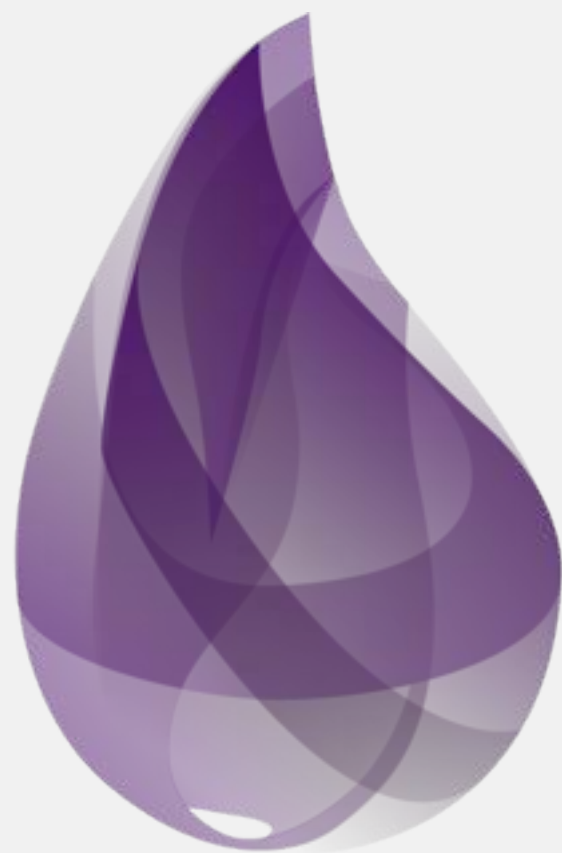
@josevalim

github.com/elixir-lang/ecto



plataformatec

tecnologia e engenharia de software



elixir

Next
generation

Topics

- Introduction to the problem
- How to use Ecto
- How Ecto works

LINQ

Language Integrated
Query

LINQ

- Part of the .NET Framework since Nov/2007
- Supported by VB, C# and F#


```
var query = from p in db.Product
              where p.ProductID > 25
              orderby p.Name
              select p;
return query.ToList();
```


Benefits

- Composability
- Data security
- Type safety

```
var query = from p in db.Product
              where p.ProductID > 25
              orderby p.Name
              select p;
return query.ToList();
```

```
var query = db.Product
    .Where(p => p.ProductID > 25)
    .OrderBy(p => p.Name)
    .Select(p => new Product(p));
return query.ToList();
```

```
var query = someCollection
    .Where(p => p.ProductID > 25)
    .OrderBy(p => p.Name)
    .Select(p => new Product(p));
return query.ToList();
```

Ecto

Language Integrated
Query for DBs

Relational algebra

Implementation

```
var query = someCollection
    .Where(p => p.ProductID > 25)
    .OrderBy(p => p.Name)
    .Select(p => new Product(p));
return query.ToList();
```



```
query = from p in Product,  
        where: p.product_id > 25,  
        order_by: p.name,  
        select: p
```

```
Repo.all(query)
```


Composability

```
query = from p in Product,  
        where: p.product_id > 25
```

```
query = from q in query  
        order_by: q.name
```

Ecto.Queue

Composability

```
product = Repo.get(Product, 1)
```

```
    from c in product.categories,  
limit: 10
```



Ecto.Que

Data Security

```
def by_name(name) do  
  from p in Product,  
    where: p.name == ^name  
end
```

Data Security

```
def names do  
  from p in Product,  
  select: strip(p.name)  
end
```

Type Safety

```
def by_name(name) do  
  from p in Product,  
    where: p.name == ^name  
end
```

Ecto

- Queries
- Entities
- Models
- Repos (Repositories)

Entities


```
IO.puts("Hello world")
```



```
defmodule Hello do
  def world() do
    IO.puts("Hello world")
  end
end
```

```
Hello.world()
```

```
defmodule Hello do
  def world do
    IO.puts "Hello world"
  end
end
```

```
Hello.world
```

```
from(p in Product,  
     where: p.product_id > 25,  
     order_by: p.name,  
     select: p)
```

```
    from p in Product,  
    where: p.product_id > 25,  
order_by: p.name,  
    select: p
```

```
defmodule MathTest do
  use ExUnit.Case, async: true

  test "basic operations" do
    assert 1 + 1 == 2
  end
end
```

```
defrecord User, [:name, :age]
```

```
user = User[name: "josé", age: 27]
```

```
user.name ==> "josé"
```

```
User[age: age] = user
```

```
age ==> 27
```

```
defmodule Product.Entity do
  use Ecto.Entity

  # id is implicit
  field :name, :string
  field :desc, :text
  field :available, :integer
end
```



```
product =  
    Product.Entity[name: "umbrella"]  
product.name
```

```
Product.Entity[available: avl] =  
    product
```


Models

```
defmodule Product do
  def available?(product) do
    # ...
  end
end
```

```
defmodule Product do
  use Ecto.Model

  def available?(product) do
    # ...
  end
end
```

Models

- **Validations**
- **Callbacks**
- **Queryable**

Validations

```
defmodule Product do
  use Ecto.Model

  validate product,
    name: present(),
    desc: has_length(5..1000),
    also: validate_attachments
end
```

Validations

```
case Product.validate(product) do
  []      -> # no errors
  errors -> # errors
end
```

Callbacks

```
defmodule Product do
  def before_create(product,
                    repo, opts) do
    # ...
  end
end
```


Callbacks

```
defmodule Product do
  use Ecto.Model
  before_create :normalize_desc
  before_create Ecto.Model.Timestamps
end
```


Queryable

```
defmodule Product do
  use Ecto.Model

  queryable "products",
            Product.Entity
end
```

```
query = from p in Product,  
        where: p.product_id > 25,  
        order_by: p.name,  
        select: p
```

```
Repo.all(query)
```

Queryable

```
defmodule Product do
  use Ecto.Model

  queryable "products",
            Product.Entity
end
```

Queryable

```
defmodule Product do
  use Ecto.Model

  queryable "products" do
    field :name, :string
    field :desc, :text
    field :available, :integer
  end
end
```

Queryable

```
defmodule User.Comment do  
  use Ecto.Model  
  queryable "comments", Comment.Entity  
end
```

```
defmodule Admin.Comment do  
  use Ecto.Model  
  queryable "comments", Comment.Entity  
end
```

Repos

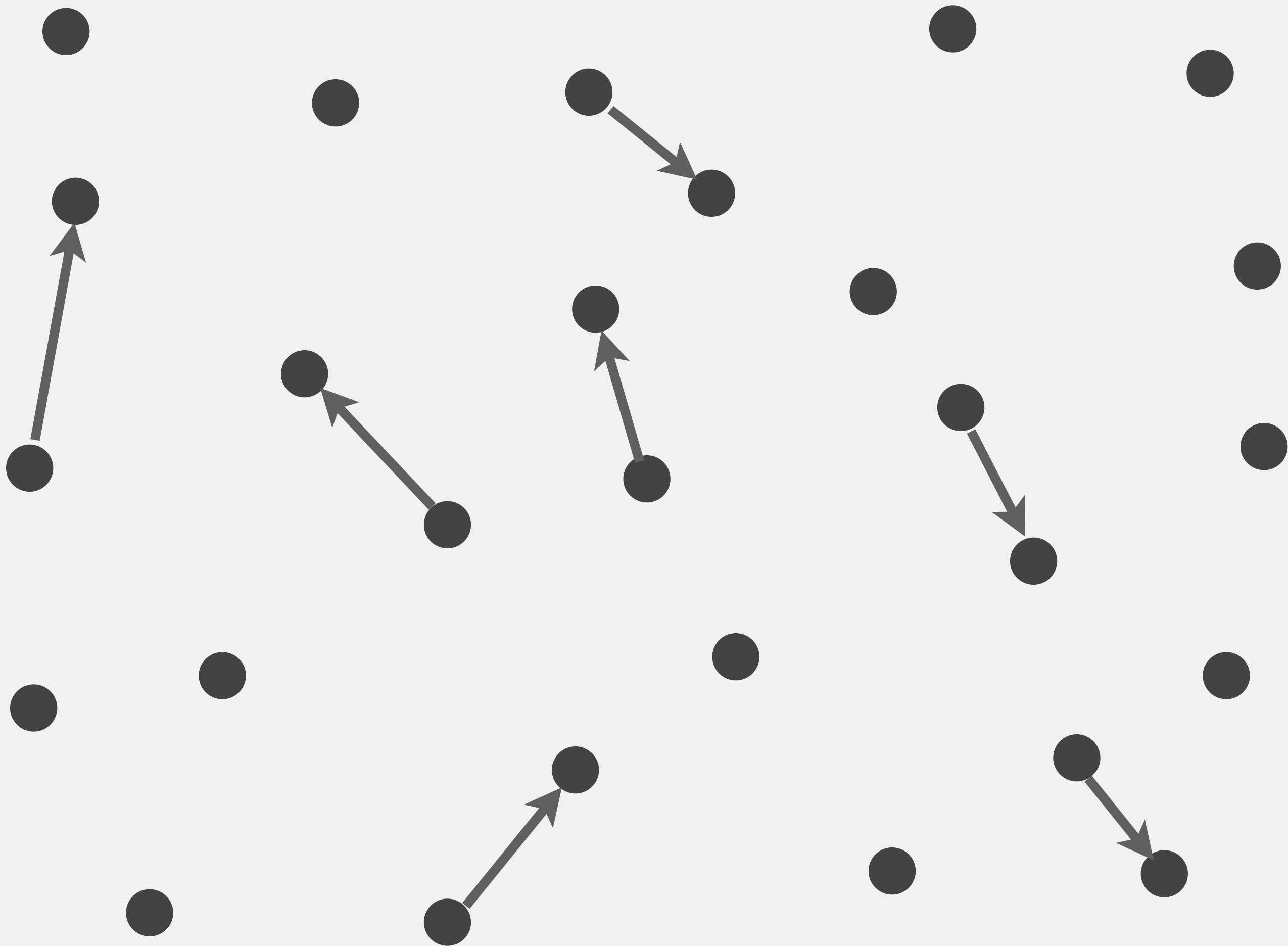
```
defmodule Repo do
  use Ecto.Repo,
    adapter: Ecto.Adapters.Postgres

  def url do
    "ecto://user:pass@localhost/sample"
  end
end
```


Repo

- `all(query)`
- `create(entity)`
- `update(entity)`
- `delete(entity)`
- `update_all(query)`
- `delete_all(query)`


```
Repo.start_link()
```



A hunky Erlang worker pool factory <http://github.com/devinus/poolboy>

 132 commits

 8 branches

 14 releases

 9 contributors



branch: master

poolboy /

Release 1.0.1



devinus authored 2 months ago

latest commit c2c939b51a

 src
Use a deadline from the calling process

2 months ago

 test
Fix wrong status

a year ago

 .gitignore
Clean up .gitignore

8 months ago

 .travis.yml
Add R16B to our Travis suite

8 months ago

 LICENSE
Make Poolboy optionally available under the Apache License

2 years ago

 Makefile
Add 'qc' to PHONY targets

8 months ago

 README.md
Separate pool args and worker args into two lists

a year ago

 UNLICENSE
Automatic checkin when parents die

3 years ago

 VERSION
Release 1.0.1

2 months ago

 package.exs
Use a deadline from the calling process

2 months ago

 package.head.exs
Use a deadline from the calling process

2 months ago

 rebar.config
Fix merge conflict and be verbose when running tests

2 years ago

Code

 Issues

5

 Pull Requests

1

 Wiki

 Pulse

 Graphs

 Network

SSH clone URL

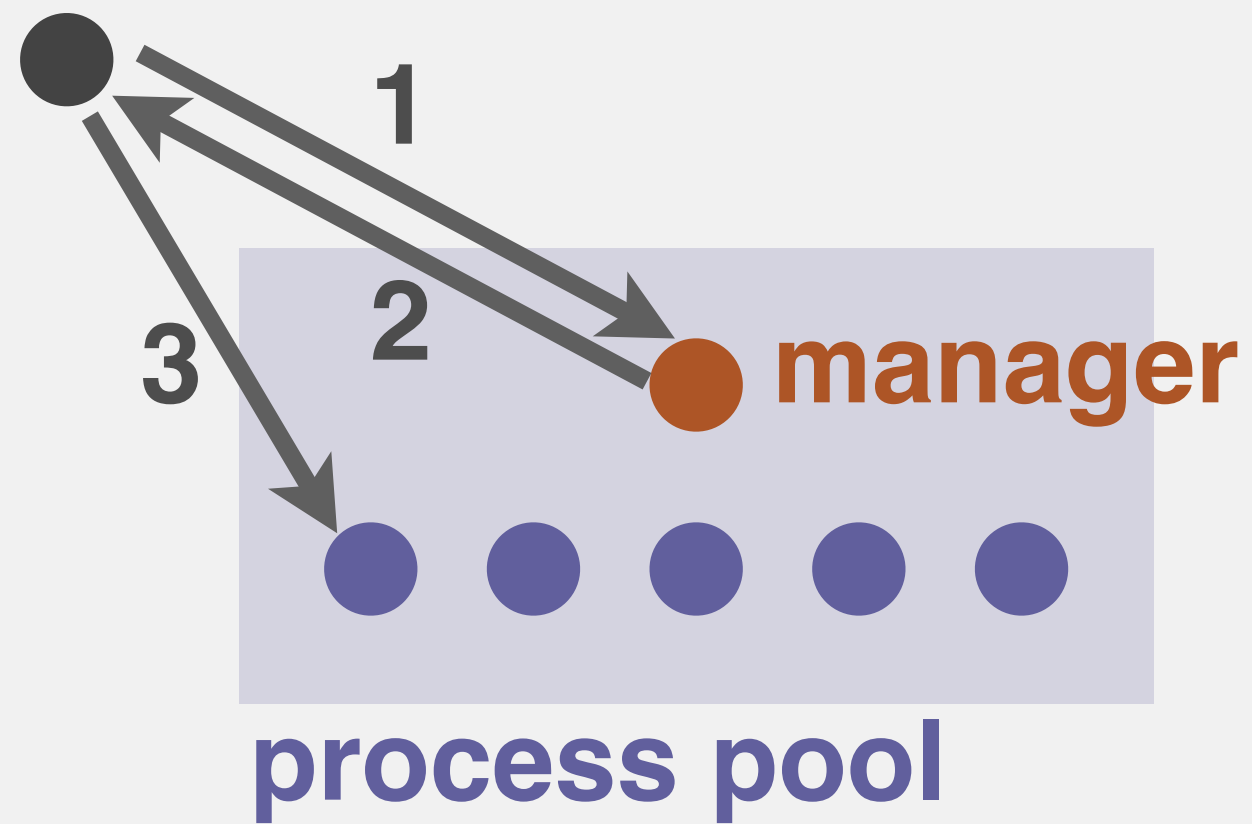
git@github.com:dev

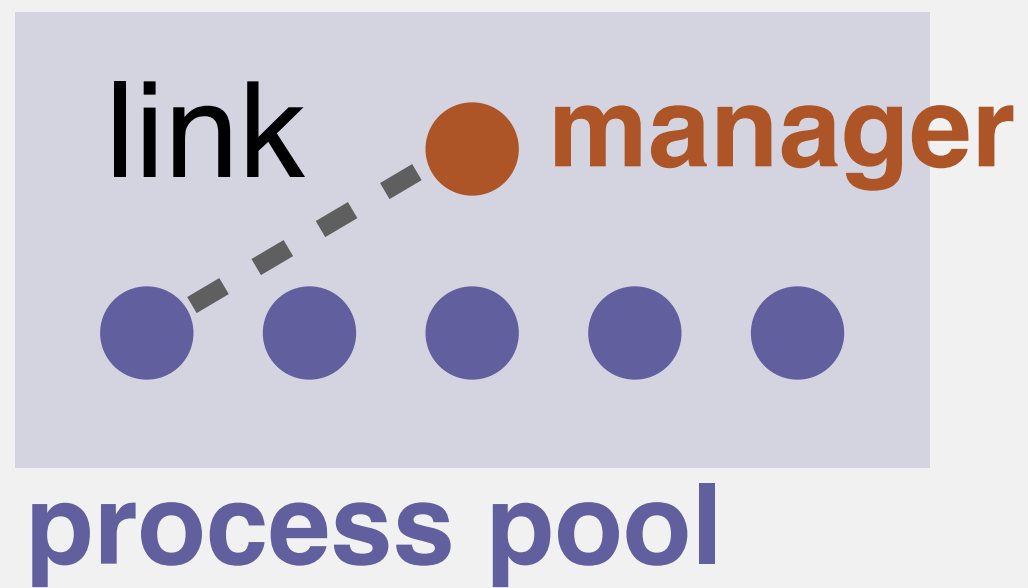


You can clone with [HTTPS](#), [SSH](#), or [Subversion](#). ?

 Clone in Desktop

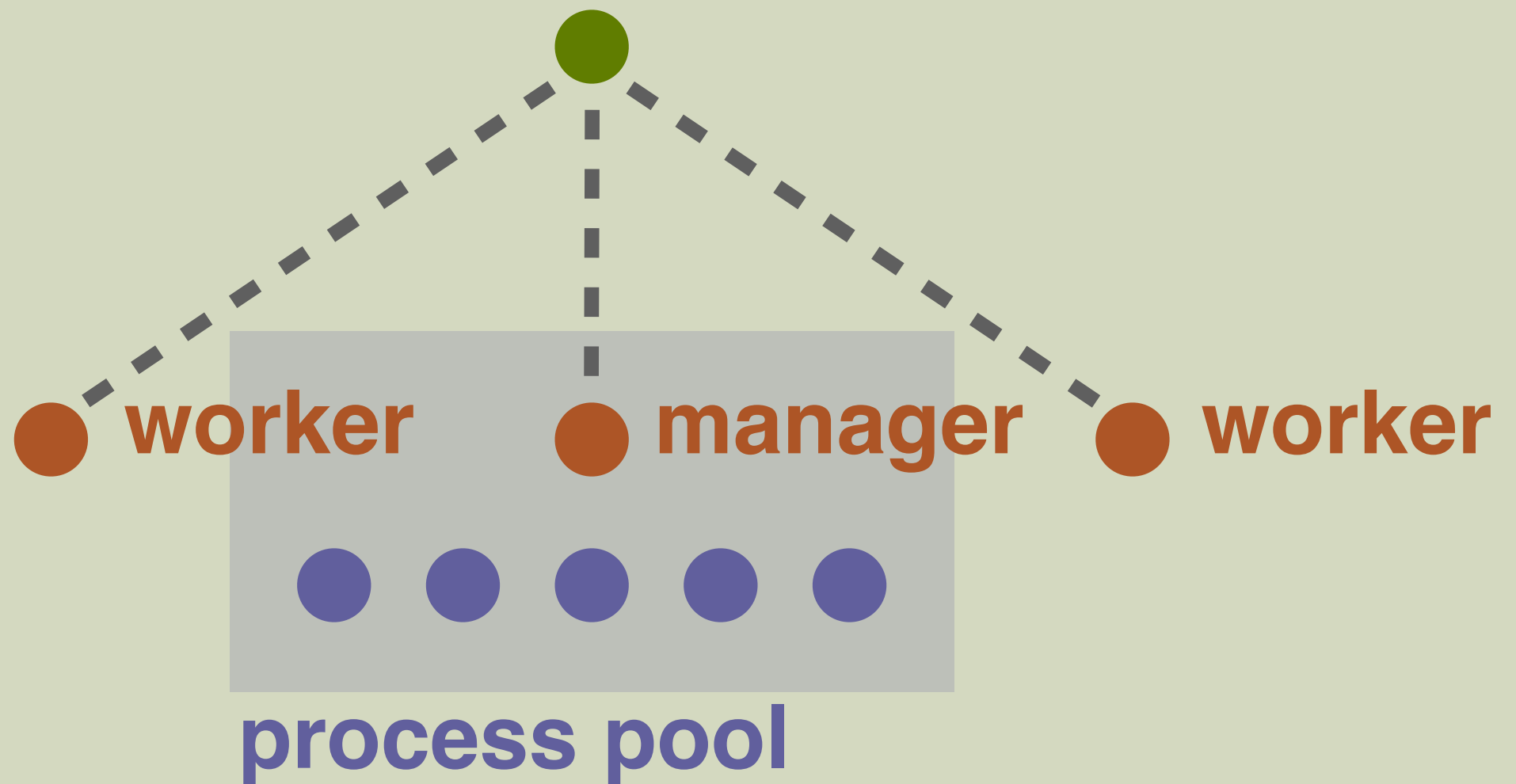
 Download ZIP





Application

App.Supervisor



App.Supervisor



Repo(.start_link)


```
defmodule App.Supervisor do
  use Supervisor.Behaviour

  def start_link do
    :supervisor.start_link(__MODULE__, [])
  end

  def init([]) do
    tree = [ worker(Repo, []) ]
    supervise(tree, strategy: :one_for_all)
  end
end
```


Ecto

- Queries
- Entities (data)
- Models (behaviour)
- Repos (storage)

Ecto

- Associations
- Generators
- Migrations



@josevalim

github.com/elixir-lang/ecto